CENTRO UNIVERSITÁRIO DO DISTRITO FEDERAL – UDF

COORDENAÇÃO DO CURSO DE SISTEMAS DE INFORMAÇÃO

Anna Beatriz Resende Oki Corrêa

Desafios da modernização de sistemas legados com agentes de IA no desenvolvimento de software

BRASÍLIA

2026

Anna Beatriz Resende Oki Corrêa

Desafios da modernização de sistemas legados com agentes de IA no desenvolvimento de software

Trabalho de conclusão de curso apresentado à Coordenação do Curso de Sistemas de Informação, do Centro Universitário do Distrito Federal - UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em Sistemas de Informação.

Orientador: Prof. Dr. Xxxx

BRASÍLIA

?Sobrenome?, ?Nome?. Título : ?author?. – Brasília, 2026. 21 f.

Orientador: ?orientador?. Trabalho de Conclusão de Curso (Graduação – Sistemas de Informação) – Centro Universitário do Distrito Federal – UDF. Coordenação do Curso de Sistemas de Informação, Brasília, DF, 2026.

1. ?Assunto?. 2. ?Assunto?. 3. ?Assunto?. I. ?Título?.

CDU: ?consultar na biblioteca?

?Titulo?

Trabalho de conclusão de curso apresentado à ?Coordenação?, do Centro Universitário do Distrito Federal – UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em ?Curso?.

Orientador: ?orientador?.

Brasília, _____ de _____ de 2026.

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

Nota: _____

Dedico este trabalho à minha família, pelo apoio, incentivo e compreensão durante toda a minha trajetória acadêmica.

Agradecimentos

Agradeço, primeiramente, a Deus, por mais uma conquista; ao meu orientador, pela dedicação e correções; e aos bibliotecários pelo suporte em todas as pesquisas.

"A educação é a arma mais poderosa que você pode usar
para mudar o mundo."

Nelson Mandela

Resumo

Este trabalho trata da atualização de sistemas legados para novas tecnologias com o uso de agentes de IA para desenvolvimento. O objetivo é identificar os desafios desse uso, do ponto de vista do desenvolvimento. Para isso, foi feita uma revisão de estudos recentes sobre o tema, e os desafios encontrados foram organizados em categorias. Também é proposto um estudo de caso em um sistema real, para observar esses desafios na prática, com avaliação por testes.

Palavras-chave: *Sistemas legados. Modernização de software. Agentes de IA. Desenvolvimento de software.*

Abstract

This work addresses the update of legacy systems to new technologies using AI development agents. The objective is to identify the challenges of this use from a development perspective. To do so, a review of recent studies on the subject was carried out, and the challenges found were organized into categories. A case study on a real system is also proposed, to observe these challenges in practice, with evaluation through tests.

Keywords: *Legacy systems. Software modernization. AI agents. Software development.*

Lista de ilustrações

Sumário

1 INTRODUÇÃO 11
1.1 Contexto 11
1.1.1 Sistemas legados 11
1.1.2 Dificuldades 12
1.1.3 Agentes de IA 12
1.2 Problema 12
1.3 Pergunta de pesquisa 13
1.4 Objetivos 13
1.5 Metodologia 13
1.6 Organização do trabalho 13

2 FUNDAMENTAÇÃO TEÓRICA 14
2.1 Sistemas legados e modernização 14
2.2 Agentes de IA no desenvolvimento 14
2.3 Conversão de código e verificação 15
2.4 Desafios encontrados na literatura 15

3 METODOLOGIA 18
3.1 O problema 18
3.2 O estudo de caso 18
3.3 Como o agente será usado 18
3.4 Como será a avaliação 19
3.5 Cuidados 19
3.6 Limitações 19

4 RESULTADOS E CONSIDERAÇÕES FINAIS 20

REFERÊNCIAS 21

1 Introdução

1.1 CONTEXTO

1.1.1 Sistemas legados

Sistemas legados são sistemas computacionais que permanecem em operação nas organizações, mesmo quando suas tecnologias, arquiteturas ou ferramentas de desenvolvimento já não acompanham as práticas atuais. Sua caracterização, entretanto, não depende exclusivamente da idade ou da tecnologia utilizada, mas também de sua importância para os processos organizacionais e das dificuldades envolvidas em sua manutenção, evolução e integração com novas soluções

Em muitos casos, esses sistemas concentram regras de negócio construídas e modificadas ao longo de anos de utilização. Parte desse conhecimento pode estar incorporada diretamente ao código-fonte, especialmente quando a documentação é incompleta ou não acompanha as alterações realizadas durante sua existência. Dessa forma, substituir ou modernizar uma aplicação legada não consiste apenas em reproduzir suas funcionalidades em uma tecnologia mais recente, mas também envolve compreender e preservar comportamentos e regras essenciais para a organização. Essa complexidade torna a migração uma atividade que exige análise e validação cuidadosas.

Estudos recentes apresentam exemplos desse cenário. No primeiro, Alves, Politowski e Montandon (2026) trabalharam com um sistema de gestão de shopping centers escrito em Visual Basic 6, uma linguagem antiga, onde o sistema estava em produção havia mais de vinte anos e era organizado de duas formas: uma em um núcleo e outra em módulos secundários, que são acrescentados conforme o pedido de cada cliente da empresa dona do sistema. Os autores escolheram dois desses módulos, mas de naturezas diferentes, o de Medições, voltado a cálculos numéricos, como consumo e tarifas, e o de Compras, que depende de transações e de fluxos de aprovação. Os dois têm cerca de 299 mil e 224 mil linhas de código e são usados continuamente pelos clientes da empresa. Eles foram migrados para C# (.NET 10), mantendo o mesmo banco de dados. A migração foi feita por um agente de IA, em uma única geração e sem feedback humano entre as tentativas, e envolveu 12 funcionalidades, seis de cada módulo: 6 simples, 4 médias e 2 complexas. O objetivo era que o código novo se comportasse como o original.

O segundo estudo é da IBM. De acordo com Chakaravarthy et al. (2026), sistemas empresariais modernos muitas vezes dependem de aplicações antigas em COBOL, uma linguagem criada há várias décadas, cuja estrutura rígida e monolítica e cujas variáveis globais dificultam a manutenção, e a adoção da nuvem. Para lidar com isso, os autores propõem traduzir o COBOL para

Java combinando a análise do código com um modelo de linguagem. A análise infere as classes e as assinaturas dos métodos em Java a partir dos dados e do fluxo de controle do programa em COBOL, e essas informações guiam o modelo na tradução da lógica dos programas. A solução faz parte de um produto da IBM, o watsonx Code Assistant for Z. Os autores a avaliaram em 20 programas de um conjunto chamado GenAPP e em 10 aplicações de clientes e, segundo eles, ela já é usada em vários ambientes de clientes da IBM.

1.1.2 Dificuldades

Esses sistemas trazem dificuldades para o desenvolvedor. No estudo de Alves, Politowski e Montandon (2026), havia regras espalhadas em vários arquivos e regras escondidas em parâmetros opcionais e em configurações globais, e elas foram as mais difíceis de migrar. Outro estudo avaliou como documentar código escrito em linguagens antigas, como MUMPS e assembly (DIGGS et al., 2024).

Por isso, atualizar um sistema legado para tecnologias novas, o que chamamos de modernização, é um trabalho que exige cuidado. O sistema novo precisa continuar fazendo o que o antigo fazia.

1.1.3 Agentes de IA

Agentes de IA para desenvolvimento são ferramentas que leem o código de um projeto, editam arquivos e rodam comandos e testes. Estudos recentes avaliaram esses agentes em tarefas de modernização (LI et al., 2026; LIU et al., 2026; MAY et al., 2025).

1.2 PROBLEMA

Usar um agente de IA não resolve tudo. Os estudos mostram limites claros. No Li et al. (2026), os agentes passaram em 91,3% dos testes em projetos pequenos, mas em apenas 15,3% nos projetos grandes. No estudo de Alves, Politowski e Montandon (2026), o agente deixou de migrar parte das regras do sistema, e as funcionalidades mais complexas foram as mais afetadas.

Isso mostra que o desenvolvedor precisa saber quais desafios aparecem quando um agente de IA atualiza um sistema legado, e como lidar com eles.

1.3 PERGUNTA DE PESQUISA

Este trabalho busca responder à seguinte pergunta:

> Quais são os desafios para converter e atualizar sistemas legados para novas tecnologias por meio de agentes de IA, do ponto de vista do desenvolvimento?

1.4 OBJETIVOS

O objetivo geral é identificar e analisar os desafios de atualizar sistemas legados com a ajuda de agentes de IA.

Os objetivos específicos são:

- levantar na literatura os desafios já relatados;
- organizar esses desafios em categorias;
- observar esses desafios em um estudo de caso com um sistema real;
- discutir o que o desenvolvedor pode fazer para lidar com cada desafio.

1.5 METODOLOGIA

O trabalho tem duas partes. A primeira é uma pesquisa na literatura sobre modernização de sistemas legados com agentes de IA. A segunda é um estudo de caso, em que um agente de IA ajuda a atualizar um sistema real.

Para avaliar o resultado do estudo de caso, a atualização só é considerada correta se o sistema compilar, rodar e passar nos testes. Essa é a mesma ideia usada por Pan et al. (2024). Os testes ficam sob controle do desenvolvedor, e o agente não pode apagá-los ou mudá-los.

1.6 ORGANIZAÇÃO DO TRABALHO

O Capítulo 2 apresenta a base teórica: sistemas legados, agentes de IA, conversão de código e os desafios encontrados na literatura. O Capítulo 3 apresenta a proposta do estudo de caso. O Capítulo 4 traz as considerações finais.

2 Fundamentação Teórica

2.1 SISTEMAS LEGADOS E MODERNIZAÇÃO

Um sistema legado é um sistema antigo que ainda está em uso. Ele pode estar escrito em uma linguagem pouco usada hoje, ter uma estrutura difícil de mudar ou depender de bibliotecas sem suporte. Mesmo assim, ele costuma ser importante, porque guarda regras de negócio que a organização usa todos os dias.

Um exemplo é o sistema estudado por Alves, Politowski e Montandon (2026). Era um sistema de gestão em Visual Basic 6, em produção havia mais de vinte anos. Outro exemplo são os programas em COBOL, que a IBM migra para Java (CHAKARAVARTHY et al., 2026).

A modernização é o trabalho de levar esse sistema para uma tecnologia mais nova, sem perder o que ele já faz. Ela pode ser de dois tipos:

- **conversão de linguagem**, por exemplo de Visual Basic 6 para C#;
- **atualização de versão**, por exemplo de Java 8 para Java 17 (LIU et al., 2026; MAY et al., 2025).

Uma dificuldade comum é a falta de documentação. Muitas vezes, o código é a única fonte do que o sistema faz. Diggs et al. (2024) testaram modelos de linguagem para documentar código legado em MUMPS e em assembly. Entender o código antigo é o primeiro passo da modernização.

2.2 AGENTES DE IA NO DESENVOLVIMENTO

Um agente de IA para desenvolvimento é uma ferramenta baseada em um modelo de linguagem que faz mais do que responder perguntas. Ele lê os arquivos do projeto, edita o código, roda comandos e testes, e repete esses passos até chegar a um resultado. O Claude Code é um exemplo.

Na modernização, o agente pode receber o código antigo e a tecnologia nova como alvo, e propor o código novo. O desenvolvedor trabalha junto: define a tarefa, revisa o resultado e decide o que aceitar.

Os agentes já foram testados nesse tipo de tarefa. Alves, Politowski e Montandon (2026) usaram o Claude Code para migrar módulos de um sistema em Visual Basic 6 para C#. May et al. (2025) e Liu et al. (2026) avaliaram agentes na atualização de projetos Java. Li et al. (2026) avaliaram agentes na modernização de repositórios em várias linguagens.

Outra linha de trabalho junta a IA com análise do código. A IBM combina análise estática do código COBOL com um modelo de linguagem para gerar código Java (CHAKARAVARTHY et al., 2026). A análise dá ao modelo informações sobre a estrutura do programa, e o resultado ficou melhor do que o de modelos usados sozinhos.

2.3 CONVERSÃO DE CÓDIGO E VERIFICAÇÃO

Converter código não é só trocar a sintaxe. O código novo precisa fazer a mesma coisa que o antigo. Isso é chamado de equivalência funcional.

Por isso, Pan et al. (2024) consideram uma tradução correta só quando o código compila, roda e passa nos testes. Compilar não basta, porque um código pode compilar e mesmo assim se comportar de forma diferente.

Os testes também têm limites. Se o sistema tem poucos testes, erros passam sem ser vistos. Por isso, o May et al. (2025) escolheu projetos com alta cobertura de testes e exige que a cobertura não caia mais de 5 pontos percentuais depois da migração. Já o Li et al. (2026) esconde os testes do agente, para que ele não os use como atalho.

Outra forma de avaliar é pedir para outro modelo de linguagem julgar o resultado. A IBM usou esse método para medir a qualidade do código Java gerado (CHAKARAVARTHY et al., 2026). Mas, no estudo de Diggs et al. (2024), as métricas automáticas tiveram pouca relação com a nota dada por pessoas, e por isso os autores indicam a avaliação humana. Neste trabalho, a verificação principal será feita com testes.

2.4 DESAFIOS ENCONTRADOS NA LITERATURA

A literatura mostra que usar agentes de IA na modernização traz resultados bons, mas também desafios. Eles estão resumidos no Quadro 1 e explicados a seguir.

Tabela 1 – Desafios da modernização com agentes de IA

Desafio	**O que os estudos mostram**	**Fonte**
Tamanho do sistema	Passou em 91,3% dos testes com menos de 10 mil linhas e em 15,3% com mais de 50 mil	Li et al. (2026)
Complexidade das funcionalidades	Equivalência de 92% (baixa), 81% (média) e 47% (alta)	Alves, Politowski e Montandon (2026)
Regras escondidas	O agente perdeu regras implícitas e regras espalhadas em vários arquivos	Alves, Politowski e Montandon (2026)
Atalhos do agente	Excluiu testes que falhavam e executou o programa original em vez de traduzi-lo	May et al. (2025); Li et al. (2026)
Linguagens antigas	Bons resultados em MUMPS, notas ruins em assembly, e alucinações em ambos	Diggs et al. (2024)
Custo	Em média, 3,81 milhões de tokens e US$ 4,19 por migração	Alves, Politowski e Montandon (2026)

Fonte: elaborado pela autora, com base nos trabalhos citados.

Tamanho e complexidade

Quanto maior e mais complexo o sistema, pior o resultado. Além dos números do Quadro 1, Liu et al. (2026) mediram 71,67% de sucesso na migração mínima de Java 8 para Java 17 e 53,33% na migração máxima. No May et al. (2025), o melhor modelo migrou 52,3% dos projetos.

Regras que o agente não vê

No estudo de Alves, Politowski e Montandon (2026), o agente deixou de migrar 100 de 331 itens analisados. A maior causa, com 42 casos, foram módulos vizinhos que ele não detectou. As regras escritas de forma explícita foram migradas melhor do que as implícitas, como as que dependem de parâmetros opcionais ou de configurações globais.

Atalhos e falsa impressão de sucesso

O agente pode parecer ter acertado sem ter acertado. No May et al. (2025), um agente excluiu testes que falhavam do processo de construção do projeto. No Li et al. (2026), em uma execução sem regras contra atalhos, um agente chegou a 99,5% de testes aprovados, mas em cinco projetos ele só chamava o programa original. Por isso, o desenvolvedor deve proteger os testes e conferir o que o agente de fato fez.

Linguagens antigas e pouca documentação

Diggs et al. (2024) observaram que os modelos tendem a inventar informações em código legado. Os comentários gerados para MUMPS foram bem avaliados, mas os de assembly tiveram notas baixas.

Custo

O custo depende do tamanho da tarefa. No estudo de Alves, Politowski e Montandon (2026), as funcionalidades de alta complexidade usaram cerca de 9 milhões de tokens, em média, e a maior parte do gasto vem do que o agente lê, e não do que escreve.

Supervisão do desenvolvedor

Em todos os trabalhos, o resultado do agente precisou de verificação. Alves, Politowski e Montandon (2026) usaram um segundo revisor para conferir as regras migradas. Diggs et al. (2024) indicam avaliação humana, porque as métricas automáticas não bastaram. E Chakaravarthy et al. (2026) mostram que juntar o modelo com análise do código melhorou os resultados e reduziu a variação entre eles.

O que ainda falta

Nos trabalhos que lemos, os estudos usam conjuntos de testes públicos ou sistemas de empresas. Falta observar esses desafios no dia a dia do desenvolvedor, em um sistema real e menor. É isso que o estudo de caso deste trabalho propõe.

3 Metodologia

3.1 O PROBLEMA

Sistemas antigos precisam ser atualizados, mas o trabalho é lento e arriscado. Um agente de IA pode acelerar esse trabalho, mas a literatura mostra desafios, como regras que o agente não vê, atalhos que ele pega e resultados difíceis de verificar (Quadro 1). O desenvolvedor precisa saber o que esperar e como se proteger.

3.2 O ESTUDO DE CASO

Para observar esses desafios na prática, este trabalho propõe um estudo de caso com um sistema real.

O sistema escolhido é **[nome do sistema]**, usado para **[finalidade]**. Ele está em **[tecnologia atual]** e será atualizado para **[tecnologia de destino]**.

O responsável pelo sistema é o *stakeholder* do trabalho. Ele autoriza o uso do sistema, explica as regras importantes e ajuda a conferir se o resultado está correto.

3.3 COMO O AGENTE SERÁ USADO

O agente de IA trabalha junto com o desenvolvedor, dentro do ambiente de desenvolvimento. O processo tem cinco passos:

1. entender o sistema: o agente lê o código e explica o que cada parte faz;
2. criar testes antes de mudar o código, para guardar o comportamento atual;
3. pedir a atualização em partes pequenas, e não o sistema inteiro de uma vez;
4. rodar os testes depois de cada parte;
5. revisar o resultado e decidir o que aceitar.

3.4 COMO SERÁ A AVALIAÇÃO

Cada parte atualizada só é considerada correta se o código compilar, rodar e passar nos testes (PAN et al., 2024). Além disso:

- os testes ficam sob controle do desenvolvedor, e o agente não pode apagá-los ou mudá-los;
- a cobertura de testes não pode cair depois da atualização, como no May et al. (2025);
- o desenvolvedor confere se o agente não pegou atalhos, como chamar o código antigo em vez de reescrevê-lo.

Durante o trabalho, cada problema encontrado é registrado e classificado de acordo com os desafios do Quadro 1. Também são anotados o tempo gasto e o uso do agente, para ter uma ideia do custo.

3.5 CUIDADOS

O código só será usado com o agente se o responsável pelo sistema autorizar por escrito. Dados sensíveis e senhas não fazem parte do estudo. Se o código não puder sair do ambiente da instituição, o estudo será feito apenas com a parte autorizada.

3.6 LIMITAÇÕES

O estudo observa um sistema, um agente e uma troca de tecnologia. Por isso, os resultados mostram o que aconteceu nesse caso e não valem para todos os sistemas. Estudos parecidos, como o de Alves, Politowski e Montandon (2026), também tratam seus resultados dessa forma.

4 Resultados e Considerações Finais

Este trabalho teve como objetivo entender os desafios de atualizar sistemas legados com agentes de IA, do ponto de vista do desenvolvimento.

Na literatura, encontramos desafios em seis pontos: tamanho do sistema, complexidade das funcionalidades, regras escondidas, atalhos do agente, linguagens antigas e custo. Em todos os estudos, o resultado do agente precisou ser verificado, e os testes foram a forma mais usada para isso.

A proposta do estudo de caso mostra como observar esses desafios em um sistema real: usar o agente em partes pequenas, proteger os testes e registrar cada problema encontrado.

Como trabalhos futuros, o mesmo método pode ser aplicado em outros sistemas, com outros agentes e outras trocas de tecnologia, para comparar os resultados.

Referências

ALVES, I. d. S. R.; POLITOWSKI, C.; MONTANDON, J. E. Legacy System Modernization with Coding Agents: A Case Study. 2026. ArXiv:2608.28972. Disponível em: <https://arxiv.org/abs/2608.28972>.

CHAKARAVARTHY, V. et al. Enterprise-scale COBOL-to-Java translation: LLMs augmented with program analysis. In: Proceedings of the 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '26). Rio de Janeiro, Brasil: [s.n.], 2026.

DIGGS, C. et al. Leveraging LLMs for Legacy Code Modernization: Challenges and Opportunities for LLM-Generated Documentation. 2024. ArXiv:2411.14971. Disponível em: <https://arxiv.org/abs/2411.14971>.

LI, X. et al. RepoMod-Bench: A Benchmark for Code Repository Modernization via Implementation-Agnostic Testing. 2026. ArXiv:2602.22518. Disponível em: <https://arxiv.org/abs/2602.22518>.

LIU, L. et al. MigrationBench: Repository-level code migration benchmark from java 8. In: Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '26). Jeju, Coreia do Sul: [s.n.], 2026. ArXiv:2505.09569.

MAY, V. et al. FreshBrew: A Benchmark for Evaluating AI Agents on Java Code Migration. 2025. ArXiv:2510.04852. Disponível em: <https://arxiv.org/abs/2510.04852>.

PAN, R. et al. Lost in translation: A study of bugs introduced by large language models while translating code. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24). Lisboa, Portugal: [s.n.], 2024.